Makemore Part 1 Feynman Talk Template

Goal: explain the lesson without reading notebook code. Keep it raw; expose weak spots.

0. One-sentence thesis

- What is this lesson doing from names.txt to generated names?
- Say it in plain language before formulas.

1. Count-based Bigram

- names.txt -> words
- add . before/after each word
- build N counts
- normalize N -> P
- sample names

2. NLL

- true bigram: current -> next
- prob = P[current, next]
- loss contribution = -log(prob)
- average over all bigrams

3. Neural Bigram

- xs/ys
- one-hot xs -> xenc
- xenc @ W -> logits
- exp + normalize -> probs
- NLL -> backward -> update W

4. Difference

- count-based: N -> P from data
- neural: W -> logits -> probs learned
- both model P(next char | current char)

Must-say checklist

- N[i,j] is count, not weight.
- P[i] is a full probability row.
- NLL punishes low probability on the correct next character.
- xenc @ W selects the current character row of W.

Page 1: Count-based Bigram

Explain how raw names become counts and probabilities.

Data flow

- names.txt -> words -> add . tokens -> adjacent pairs -> count matrix N -> probability matrix P

N: count matrix

- shape: 27 x 27
- row i = current char
- col j = next char
- N[i,j] = count(i -> j)

P: probability matrix

- P = normalize each row of N
- P[i,j] = P(next=j | current=i)
- each row sums to 1

Toy example space

- Use toy_words = ["ab", "aa"]
- Write bigrams, N, and P[a] by hand.

Common traps

- Do not call N random weights.
- Do not treat P[a] as one number; it is a row distribution.

Page 2: Negative Log Likelihood

Explain what the loss punishes.

Core idea

- For each real bigram current -> true_next:
- prob = model probability assigned to true_next
- loss contribution = -log(prob)

High probability

- P(true next | current) = 0.8
- -log(0.8) is small
- small penalty

Low probability

- P(true next | current) = 0.01
- -log(0.01) is large
- large penalty

Derive in words

- Why multiply probabilities for likelihood?
- Why log turns product into sum?
- Why negative makes it a minimization loss?
- Why average over all bigrams?

Must say

- NLL punishes low probability assigned to the correct next character.

Page 3: Neural Bigram

Explain how W learns the same conditional distribution.

Pipeline

- xs, ys -> one-hot xs -> xenc
- xenc @ W -> logits
- exp + row normalize -> probs
- correct probs -> NLL loss
- backward -> update W

Shapes

- xs: (num,)
- xenc: (num, 27)
- W: (27, 27)
- logits/probs: (num, 27)

Key intuition

- one-hot current char has one 1
- xenc @ W selects one row of W
- that row gives 27 next-char logits

Loss and update

- probs[range(num), ys] = probabilities of correct next chars
- loss = -log(correct_probs).mean()
- W.grad tells how to change W
- W += -lr * W.grad

Final comparison

- Count-based learns P from counts. Neural bigram learns P through trainable W.